

Research Assignment

1. Question

1. **Why is workflow/process analysis important before developing a software system? Explain how process understanding affects software functionality and user experience.**

Because before building any software, you need a blueprint or diagram. Process analysis means figuring out exactly how a system works in real life before writing any code. If you don't understand the real-world process, you risk building a technically flawless system that completely misses the mark for the user.

- **Software functionality:** If you don't understand the process first, you will build code that misses important features or has broken logic.
- **User experience:** If you don't understand the user's steps, the software will be confusing and hard to use.

2. **How can activity diagrams help identify problems in a system process before implementation? Discuss issues such as missing steps, repeated activities, and incorrect decisions or inefficient workflow.**

Activity diagrams are like flowcharts, they are used to visualize workflows and processes step-by-step. Because they let you see the whole process, it is easy to catch mistakes before spending time and money coding them.

- **Missing steps:** You can easily spot if a step is forgotten or missing. For example, in a food delivery system, if the process moves directly from receiving an order to preparing food without checking payment confirmation, the restaurant may prepare unpaid orders. The Activity Diagram helps reveal this missing process.
- **Repeated activities:** If the user has to do the same thing twice unnecessarily, like entering their email address during signup and then again before checkout, the diagram makes this waste of time obvious. For example, if users must repeatedly log in or enter personal information during one transaction, the process becomes inefficient and inconvenient. Visualizing the workflow makes these repeated actions easier to detect.

- **Incorrect decisions/Inefficient workflow:** The diamond shape shows choices. If a “no” path leads to a dead end instead of taking the user back to the previous page, the diagram will expose that flaw instantly. In an ATM system, for example, the system should stop a withdrawal if the account balance is insufficient. If the Activity Diagram shows an incorrect decision flow, developers can fix the problem before implementation.
3. **Many software systems fail because of poor process flow rather than coding errors. Do you agree with this statement? Why or why not?**

I agree because while you can write perfect code, if the app forces the user through a broken or confusing process, the software will fail. It's easier to fix a coding bug than a broken system design. Although programming mistakes can create technical problems, workflow and process design are often more important to the overall success of a system. A software system may contain excellent code, but if the process itself is confusing, inefficient, or poorly organized, users may still reject the system. Coding errors are usually easier to identify and fix through testing and debugging. However, problems related to workflow design can affect the entire system structure and user experience. If developers misunderstand business requirements or create inefficient workflows, the software may fail to solve the actual problem it was designed for.

Real-World Example: [Healthcare.gov](https://www.healthcare.gov) (2013 Launch)

When the US government launched this health insurance website, it immediately crashed. The main issue wasn't just bad coding, it was a bad process flow. The system forced millions of users to create an account and verify their identity before they could even look at insurance prices, thus resulting in the issue of having too many people hit the identity database at the exact same time, overloading the servers and crashing the site.

4. **Choose a real-world system (ATM, food delivery, online shopping, hospital, school registration, etc.) and analyze the main process/workflow, possible bottlenecks or workflow problems, and how Activity diagrams could improve the process efficiency.**

Online shopping system:

Workflow

1. User browse items and add items to cart
2. User proceed checkout
3. User enter shipping info
4. System calculates total price and shipping fee
5. User enters payment info
6. System checks payment and confirms order **Possible bottlenecks:**
 - **Slow payment:** If the bank takes too long to process the card, the user gets stuck on the loading screen and might have to leave the site.
 - **Out of stock item:** If an item sells out while a user is actively trying to buy it, the system might accidentally take their money.

By being able to map this out with visual lanes for the customer, system, and the bank, developers can see exactly where the process slows down, thus being able to help them design features to handle errors cleanly.

5. How do decisions and branching processes affect system behavior in activity diagrams? Explain why decision points are important in workflow modeling with real case.

Decisions and branching make software dynamic, they control how the system reacts under different conditions. Without them, software would be a straight line where every user experiences the exact same steps. Decisions allow the system to choose a path based on conditions, like Yes or No. In Activity Diagrams, decision nodes determine which path the workflow should follow based on conditions.

Real Case Example: ATM Withdrawal System

Workflow:

1. User inserts ATM card
2. User enters PIN
3. System checks PIN validity

1st Decision Point:

- If PIN is correct → continue transaction
- If PIN is incorrect → display error message

2nd Decision Point:

- If account balance is sufficient → dispense cash
- If insufficient → reject withdrawal

These decisions are very important because they protect customer accounts, prevent unauthorized access, and ensure accurate transactions.

Without proper branching logic, the ATM could allow unauthorized users, dispense money incorrectly, and cause banking errors.